Transformations, cameras and curves

CS405 Project 1 · Arda Dinç
Şeker Ahmed Paşa · Kavunlu Natürmort · Sakıp Sabancı Museum

Repository: github.com/arddnc/cs405-project1 · local tag: project1-final

1. Painting and scene

The melon slice provides a clear organic contour for a piecewise curve. Its long lower rim makes tangent continuity visible, while the painted tabletop suggested the table, melon and articulated lamp placed in front of the painting.

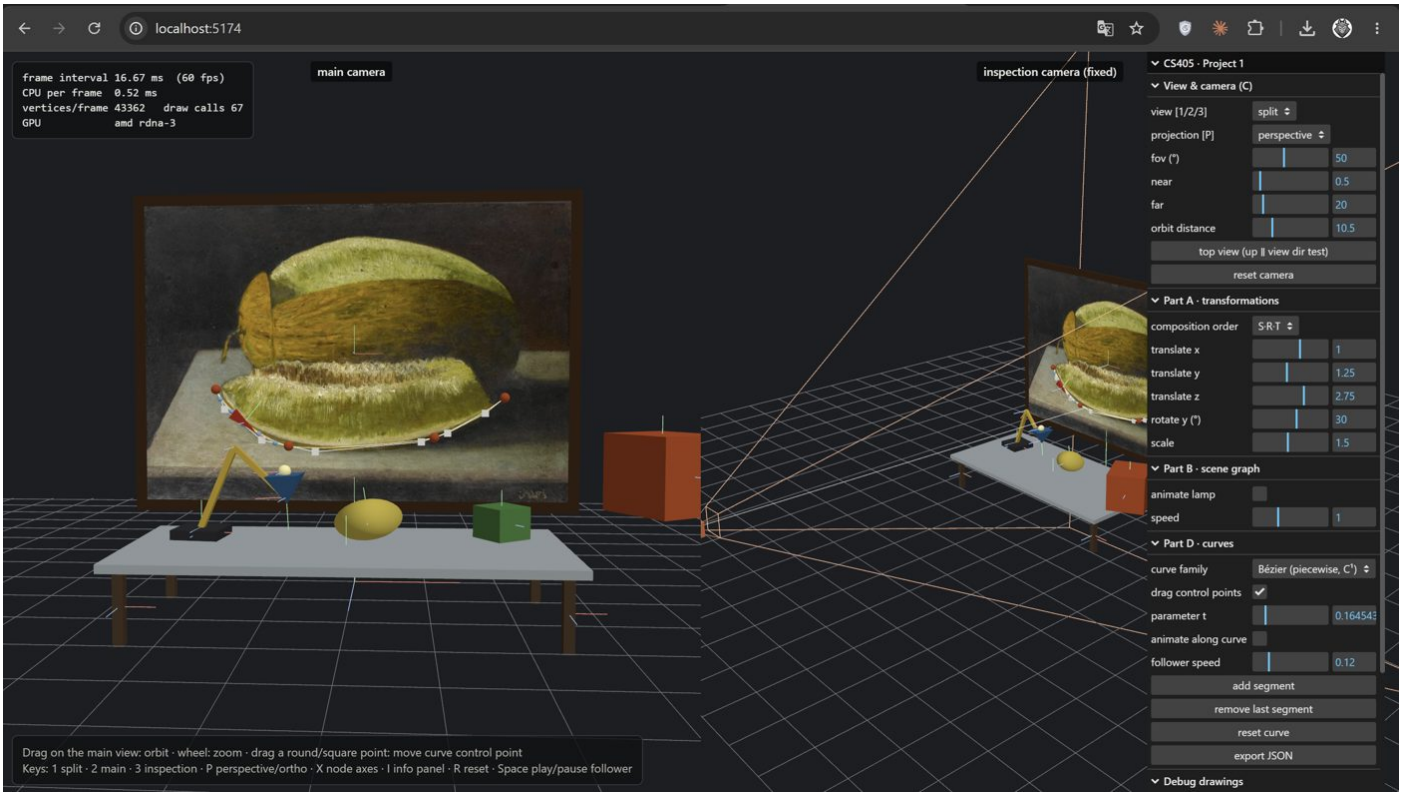


Figure 1. Complete scene in split view; the fixed inspection camera reveals the main camera and its frustum.

2. Part A - Transformation order

The project uses column vectors, column-major storage (index = 4c + r), and T·R·S as its normal model convention. Thus M p = T(R(S p)): scale about the local origin, rotate, then translate. S·R·T applies translation first, so rotation and scale also change the offset. The controlled object intentionally offers both orders.

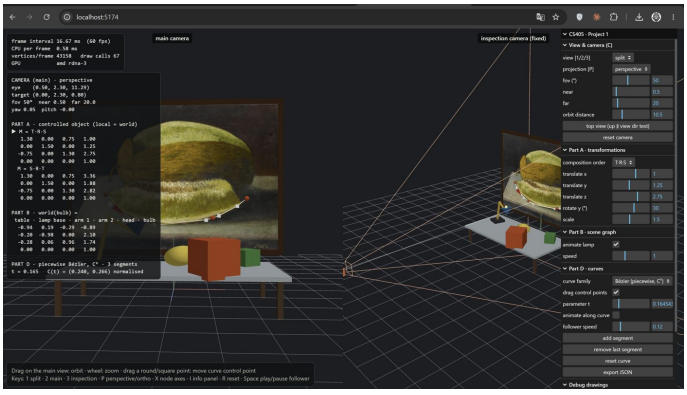


Figure 2a. T·R·S, scale 1.5.

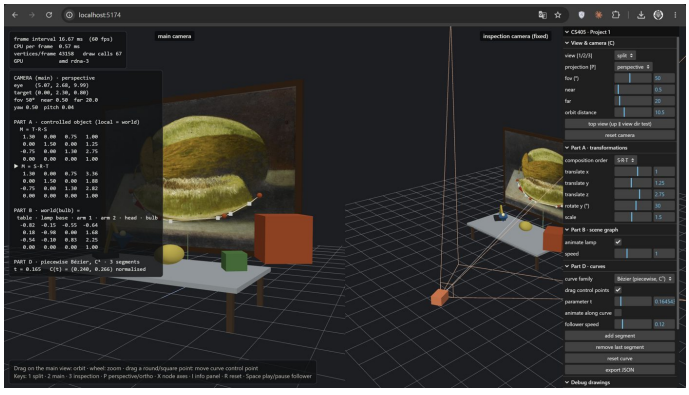


Figure 2b. S·R·T, scale 1.5.

With t = (1, 1.25, 2.75), rotation y = 30° and s = 1.5, the last column changes from **(1, 1.25, 2.75)** to **(3.36154, 1.875, 2.82235)**, matching the displayed (3.36, 1.88, 2.82). Camera angles differ between these captures; the printed matrices provide the direct comparison.

3. Part B - Hierarchical transforms

Each node computes world = parentWorld · local and passes that result to its children. Mesh sizing is stored separately in shape.matrix and is not inherited. Animated joints are the lamp base, arm 1, arm 2 and head.

```
root
├─ painting ─┬─ frame
│            └─ curve follower
├─ table ────┬─ 4 legs
│            ├─ lamp base → arm 1 → arm 2 → head → bulb
│            ├─ melon
│            └─ box
└─ controlled object
```

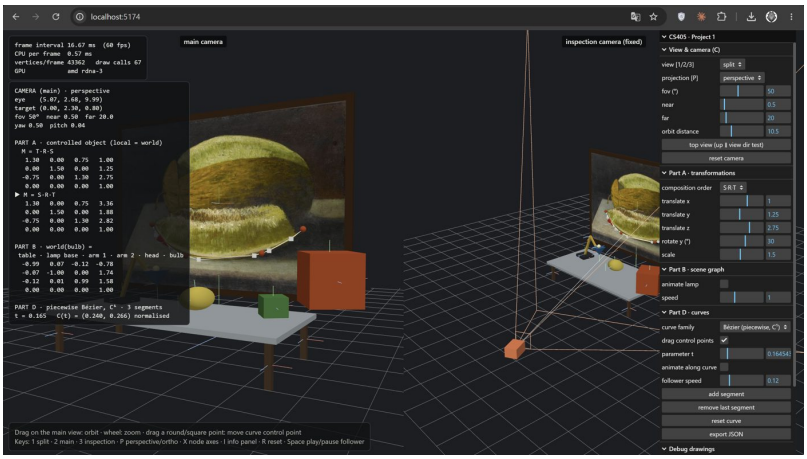


Figure 3. Paused lamp with node axes enabled.

```
Displayed world(bulb):
[-0.99  0.07 -0.12 -0.78]
[-0.07 -1.00  0.00  1.74]
[-0.12  0.01  0.99  1.58]
[ 0.00  0.00  0.00  1.00]
```

Translation-column verification

W = M_table · M_base · M_arm1 · M_arm2 · M_head · M_bulb. The screenshot does not record joint angles or animation time. The values below reconstruct one compatible pose from the source animation and the rounded matrix; they are not a claim to have recovered the exact paused timestamp. All 12 displayed upper entries agree within 0.005.

Local transforms: M_table = T(0, .85, 1.9); M_base = T(-1.85, .11, -.45) Ry(β); M_arm1 = T(0, .06, 0) Rz(α); M_arm2 = T(0, 1.1, 0) Rz(δ); M_head = T(0, .9, 0) Rz(η); M_bulb = T(0, -.32, 0).

Compatible angles (radians): β = -0.119063, α = -0.406315, δ = -1.904237, η = -0.763530.

```
p0 = bulb translation                (0.00000, -0.32000, 0.00000)
p1 = Rz(η)p0 + (0, .9, 0)            (-0.22127, 0.66883, 0.00000)
p2 = Rz(δ)p1 + (0, 1.1, 0)           (0.70442, 1.09018, 0.00000)
p3 = Rz(α)p2 + (0, .06, 0)           (1.07793, 0.78302, 0.00000)
p4 = Ry(β)p3 + (-1.85, .11, -.45)    (-0.77970, 0.89302, -0.32196)
p5 = p4 + (0, .85, 1.9)              (-0.77970, 1.74302, 1.57804)
```

Each planar rotation uses Rz(θ)(x,y,z) = (x cosθ - y sinθ, x sinθ + y cosθ, z). The final position rounds to **(-0.78, 1.74, 1.58)**, the translation column in Figure 3. This checks the hierarchy and multiplication order at screenshot precision; full-precision capture would be needed for an exact original-frame check.

4. Part C - View and projection

View matrix. Let w = normalize(eye - target), u = normalize(up × w), v = w × u. The camera-to-world matrix is C = [R | eye], with R columns u, v, w. Because R is orthonormal, V = C^(-1) = [R^T | -R^T eye]. Its rows are u, v, w; translation is (-u·eye, -v·eye, -w·eye). If |up × w| < 10^(-6), use (0,0,1), or (1,0,0) if that is also parallel. The top-view capture demonstrates a finite view at the pole.

Perspective depth. For z_ndc = (Az + B)/(-z), z = -n gives -An + B = 0, so B = An. At z = -f: -Af + B = f. Substitution yields A(n - f) = f, hence A = f/(n - f), B = nf/(n - f). With g = cot(fovY/2), the matrix rows are:

```
[g/aspect  0  0  0]    [0  g  0  0]    [0  0  A  B]    [0  0 -1  0]
```

This maps depth to [0,1], unlike the conventional OpenGL [-1,1] range. Orthographic rows are [2/(r-l),0,0,-(r+l)/(r-l)], [0,2/(t-b),0,-(t+b)/(t-b)], [0,0,1/(n-f),n/(n-f)], [0,0,0,1]. No perspective divide changes scale because clip w = 1.

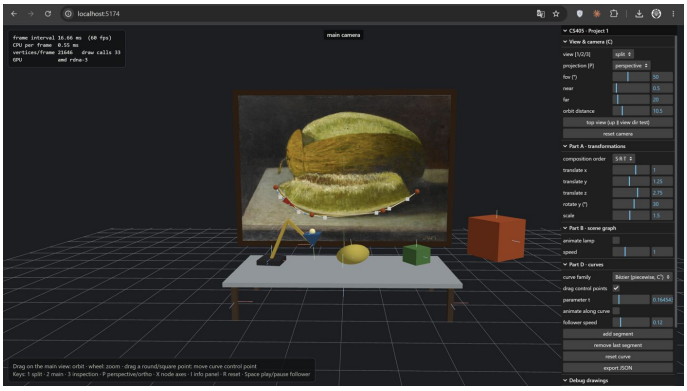


Figure 4a. Perspective, near .5 / far 20.

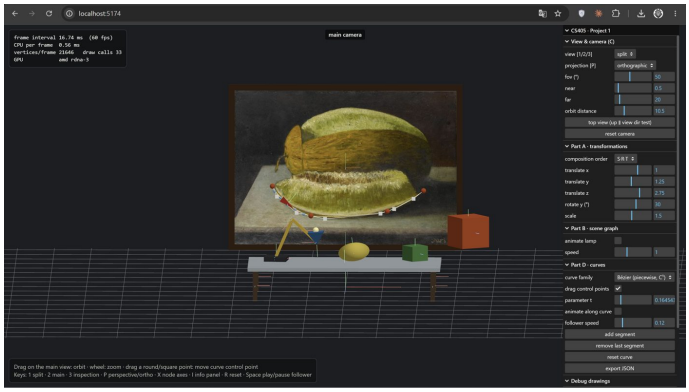


Figure 4b. Orthographic, same viewpoint.

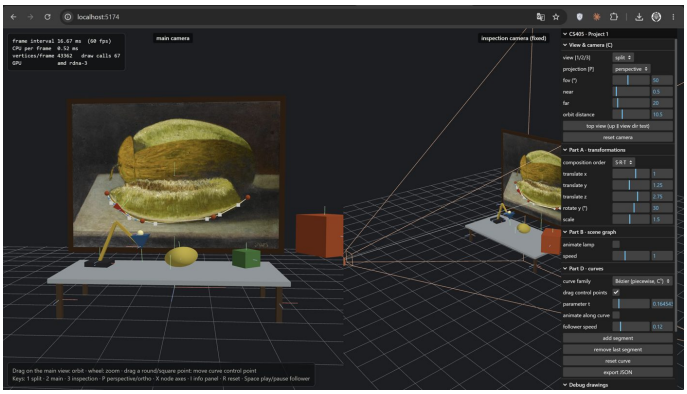


Figure 4c. Perspective frustum in inspection view.

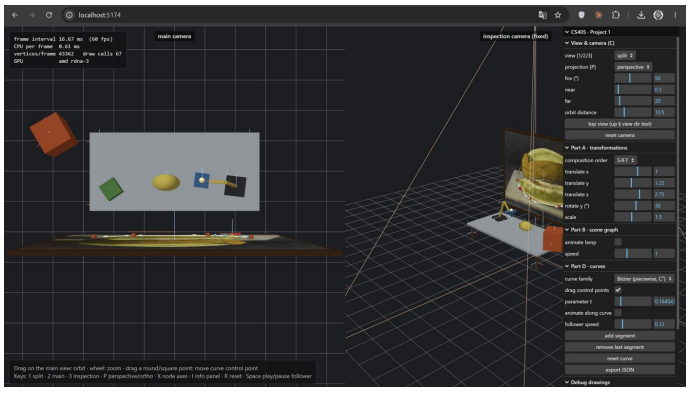


Figure 4d. Top view: parallel-up fallback.

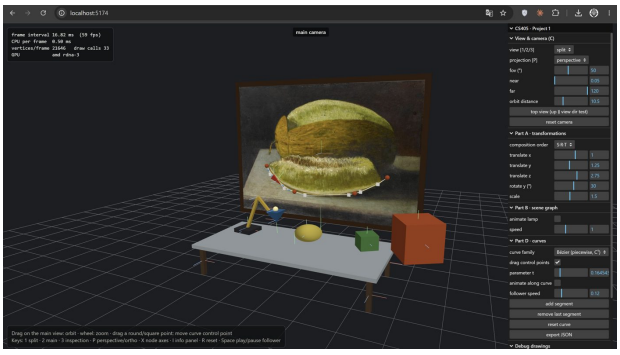


Figure 4e. Near .05 / far 120 experiment.

Near/far experiment. The usual range is n = .5, f = 20 (ratio 40). The test uses n = .05, f = 120 (ratio 2400). Perspective depth varies with 1/z, concentrating resolution near the camera. A smaller near plane worsens distant depth separation. No obvious z-fighting is visible in this captured frame; that does not prove precision is unchanged or rule out flicker during motion.

5. Part D - Tracing the melon slice

The exported curve contains three cubic Bézier segments and ten control points in normalized painting coordinates (x right, y up). The knots and handles were manually adjusted in the application to trace the lower rim. The captured comparison keeps the same viewpoint and knots when switching curve family.

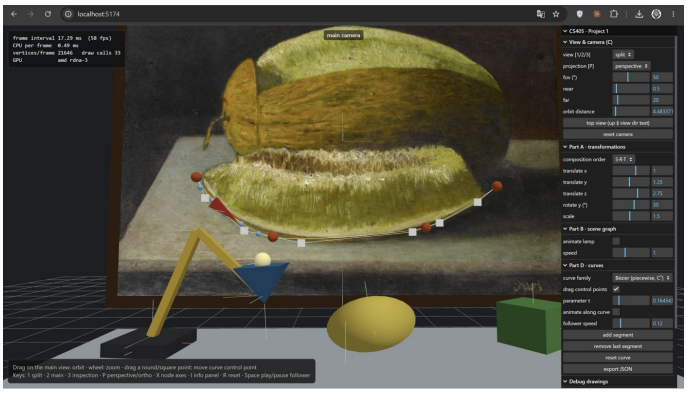


Figure 5a. Edited Bézier with de Casteljau construction.

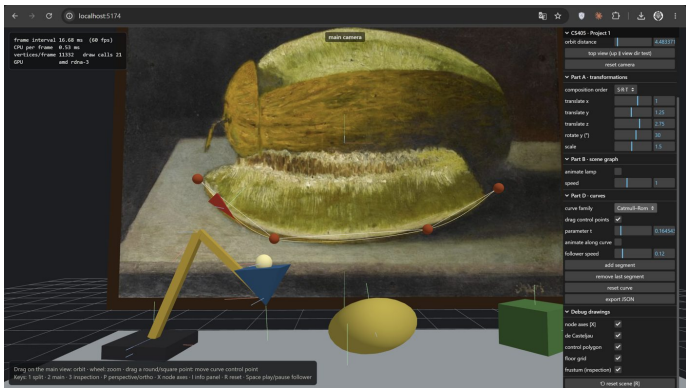


Figure 5b. Catmull-Rom through the same knots.

Bernstein basis: $B_0 = (1-t)^3$, $B_1 = 3t(1-t)^2$, $B_2 = 3t^2(1-t)$, $B_3 = t^3$. The curve is $B(t) = \Sigma\, B_i(t)P_i$. At t = 0 the weights are (1,0,0,0), giving $B(0) = P_0$; at t = 1 they are (0,0,0,1), giving $B(1) = P_3$.

t	B0	B1	B2	B3
0	1	0	0	0
0.25	.421875	.421875	.140625	.015625
0.50	.125	.375	.375	.125
0.75	.015625	.140625	.421875	.421875
1	0	0	0	1

C¹ continuity. $B'(1) = 3(K - P_2)$ and $C'(0) = 3(Q_1 - K)$. Equating them gives $Q_1 = 2K - P_2$: opposite handles mirror about the knot. The JSON rounds to five decimals, leaving at most 0.00001 component residual in this relation. De Casteljau repeatedly interpolates neighboring points, producing levels of 4, 3, 2 and 1 points; its last point equals the Bernstein result.

Catmull-Rom. The uniform segment interpolates P_1 at t = 0 and P_2 at t = 1, with tangent $(P_{i+1} - P_{i-1})/2$ at an interior knot. This application duplicates endpoint knots. Unlike Bézier handles, all four displayed knots lie on the curve; the different lower-rim shape is visible in Figure 5b.

6. What went wrong and verification

PowerShell refused to run npm (execution policy blocked npm.ps1); I fixed it with Set-ExecutionPolicy -Scope CurrentUser RemoteSigned. My first commit failed because Git had no user identity; I set user.name and user.email. A first version of src/math was generated by Codex; because that is not allowed for graded maths, I deleted those bodies and rewrote all of src/math and Node.updateWorld myself, using the 28 unit tests to check each file (vec3, mat4, camera, curves). The GPU read-out showed AMD while an NVIDIA GPU is also installed; Task Manager confirmed Chrome uses the AMD Radeon 890M.

7. Performance

Chrome / WebGPU; AMD Radeon 890M Graphics (application: “amd rdna-3”). Lamp and follower paused, Bézier and debug drawings enabled. These are screenshot samples, not a repeated benchmark. CPU time measures frame-loop work; GPU execution time was not measured.

View	Interval	FPS	CPU	Vertices	Draw calls
Split	17.07 ms	59	0.48 ms	43,362	67
Single	16.67 ms	60	0.48 ms	21,646	33

Split view roughly doubles submitted geometry and draw calls. Both samples are near a 60 Hz frame cadence, consistent with display pacing; they do not establish maximum throughput. Evidence: screenshots/perf.png, perf_single.png and gpu_amd.png. NVIDIA RTX 5070 Laptop GPU is also installed, but is not the adapter reported by this application.

Appendices - Provenance and curve data

A. AI use

Tool / date	What I asked	What I kept
Claude / 8 Nov 2026	Explanation of the requirements and of the maths behind each function; debugging help while I rewrote src/math.	Explanations only.
Claude / 8 Nov 2026	Generate the application scaffold (no starter repo): WebGPU renderer, WGSL shaders, geometry, scene layout, camera/curve state, picking and dragging, GUI, unit tests, README, report outline.	Kept as generated.
Codex / 8 Nov 2026	Implement the function bodies in src/math (first version).	Discarded: I deleted them and rewrote all of src/math and Node.updateWorld myself before submission; the submitted maths is my own.
Codex / 8 Nov 2026	Draft the report text, the PDF build script docs/build_report.py and the Part B pose reconstruction (tmp/pdfs/matrix_check.json).	Kept with my edits; Section 6 and this appendix rewritten by me.

My contribution: all of src/math and Node.updateWorld (rewritten by me), the traced curve and its export, scene settings and all screenshots.

B. Exported curve (curve.json)

Complete file contents below, compacted only for layout; values and fields are unchanged.


```
{
  "painting": {
    "artist": "Şeker Ahmed Paşa",
    "title": "Kavunlu Natürmort",
    "collection": "Sakıp Sabancı Museum"
  },
  "shape": "lower rim of the cut melon slice",
  "coordinates":
    "normalised painting coordinates: x right, y up, (0,0) = bottom-left, (1,1) = top-right",
  "type": "piecewise cubic Bezier (C1)",
  "segments": 3,
  "controlPoints": [
    {"index": 0, "role": "knot", "x": 0.17027, "y": 0.37916},
    {"index": 1, "role": "handle", "x": 0.1911, "y": 0.31143},
    {"index": 2, "role": "handle", "x": 0.27986, "y": 0.20668},
    {"index": 3, "role": "knot", "x": 0.3436, "y": 0.18696},
    {"index": 4, "role": "handle", "x": 0.40735, "y": 0.16723},
    {"index": 5, "role": "handle", "x": 0.65732, "y": 0.20242},
    {"index": 6, "role": "knot", "x": 0.68803, "y": 0.2172},
    {"index": 7, "role": "handle", "x": 0.71873, "y": 0.23198},
    {"index": 8, "role": "handle", "x": 0.80605, "y": 0.29908},
    {"index": 9, "role": "knot", "x": 0.85122, "y": 0.35123}
  ],
  "knots": [
    {"x": 0.17027, "y": 0.37916},
    {"x": 0.3436, "y": 0.18696},
    {"x": 0.68803, "y": 0.2172},
    {"x": 0.85122, "y": 0.35123}
  ]
}
```


Supporting evidence: the screenshots directory preserves all 14 original captures at their original resolutions. The mathematical reconstruction is reproducible from docs/build_report.py; its intermediate values are in tmp/pdfs/matrix_check.json.